

Parallel I/O

Lecture 22

April 6, 2026

Parallel Read using Explicit Offset

MPI_Offset offset = (MPI_Offset) rank*file_size_per_proc*sizeof(int)

MPI_File_open (MPI_COMM_WORLD, “/scratch/largefile”,
MPI_MODE_RDONLY, MPI_INFO_NULL, &fh)

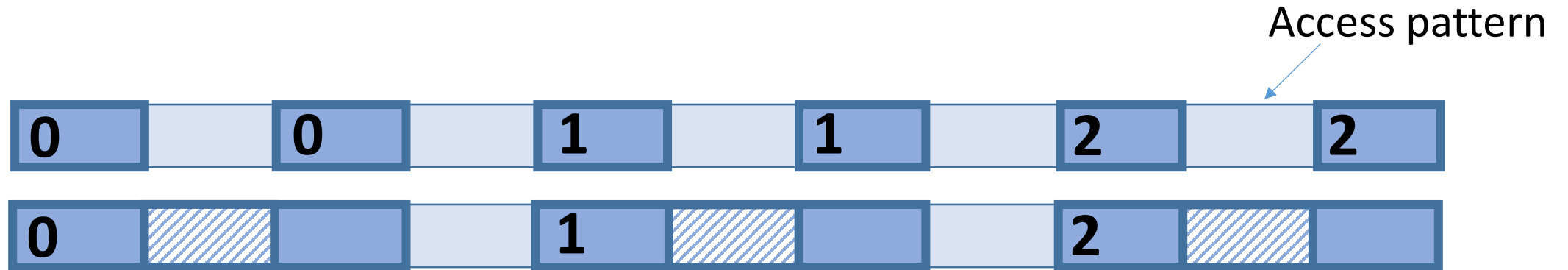
MPI_File_read_at (fh, offset, buffer, count, MPI_INT, status)

MPI_File_close (&fh)

Every process reads a **contiguous** chunk



Optimization – Data Sieving



`MPI_File_read_at` (fh, offset1, buffer1, count1, MPI_INT, status)

`MPI_File_read_at` (fh, offset2, buffer2, count2, MPI_INT, status)

`MPI_File_read_at` (fh, offset3, buffer3, count3, MPI_INT, status)

- Make large I/O requests and extract the data that is really needed
- Huge benefit of reading large, contiguous chunks

P0 P1 P2

0	1	2	0	1	2	
3	4	5	3	4	5	
6	7	8	6	7	8	
P3			P4			P5
P6			P7			P8

0 1 2 3 4 5 6 7 8

Every process in this example owns a 3x3 sub domain which may be stored as a 1D array as shown here. The first row elements of P0 will be followed by the first row elements of P1 and so on in order to maintain the correct row-wise information of the entire domain in a file.

Non-contiguous Access Patterns



Multi-variable Dataset

Two ways to store multiple fields/variables

- Var1 for the entire domain, followed by Var2 for the entire domain, followed by Var3 for the entire domain and so on
- Var1, Var2, Var3, ... for grid point 1, Var1, Var2, Var3, ... for grid point 2 and so on

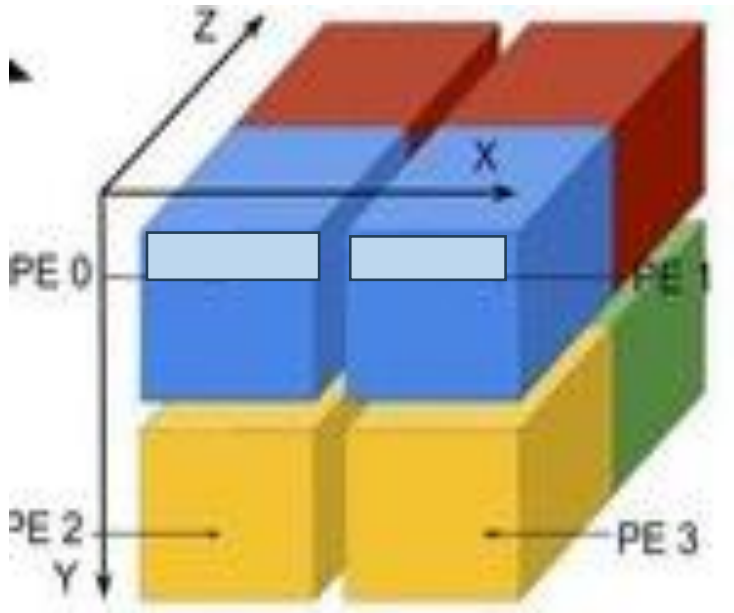
Every process analytically computes the offsets in both cases

HW: Write the code to write 4 variables of a $M \times N$ domain from $P \times Q$ processes using the above two approaches.

Non-contiguous I/O

Domain: $N \times N \times N$

Process decomposition: $P \times P \times P$



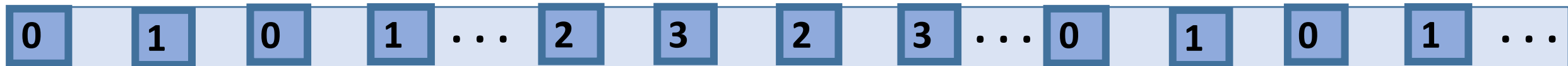
//Data written in XYZ order

//set the proper offset

for ? times

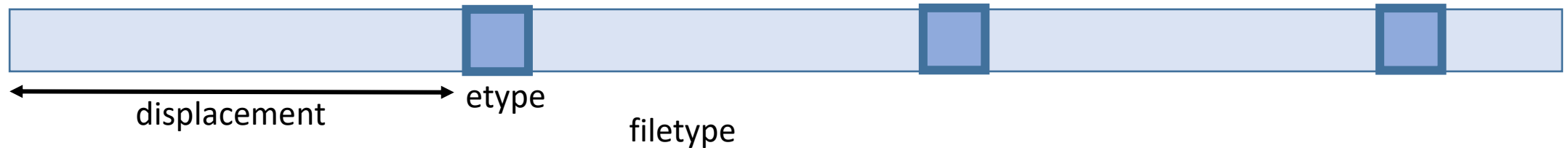
MPI_File_write_at (...)

MPI_File_write_at (...)



File View

Non-contiguous access pattern can be specified using a view



Each process can specify a view

displacement
etype
filetype

} File view

```
int MPI_File_set_view(MPI_File  
fh, MPI_Offset disp,  
MPI_Datatype etype, MPI_Datatype  
filetype, ROMIO_CONST char  
*datarep, MPI_Info info)
```

MPI_File_set_view (fh, disp, etype, filetype, "native", MPI_INFO_NULL)

MPI_File_read (fh, buffer, count, MPI_INT, status)

Independent I/O - Set File View

```
MPI_File_open (MPI_COMM_WORLD, filename, MPI_MODE_RDWR | MPI_MODE_CREATE, MPI_INFO_NULL, &myfile);

for (i=0; i<BUFSIZE; i++) {
    buf[i] = myrank + i;
}

// File write - set process view
MPI_File_set_view(myfile, myrank * BUFSIZE * sizeof(int), MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_File_write (myfile, buf, BUFSIZE, MPI_INT, MPI_STATUS_IGNORE);

// File read - set process view
MPI_File_set_view(myfile, myrank * BUFSIZE * sizeof(int), MPI_INT, MPI_INT, "native", MPI_INFO_NULL);
MPI_File_read (myfile, rbuf, BUFSIZE, MPI_INT, MPI_STATUS_IGNORE);

MPI_File_close (&myfile);

for (i=0; i<BUFSIZE; i++) {
    if (buf[i] != rbuf[i]) printf ("%d %d %d\n", i, buf[i], rbuf[i]);
}
```

0	1	2	3	4	5
---	---	---	---	---	---

Non-contiguous I/O

MPI_File_write_at (...)

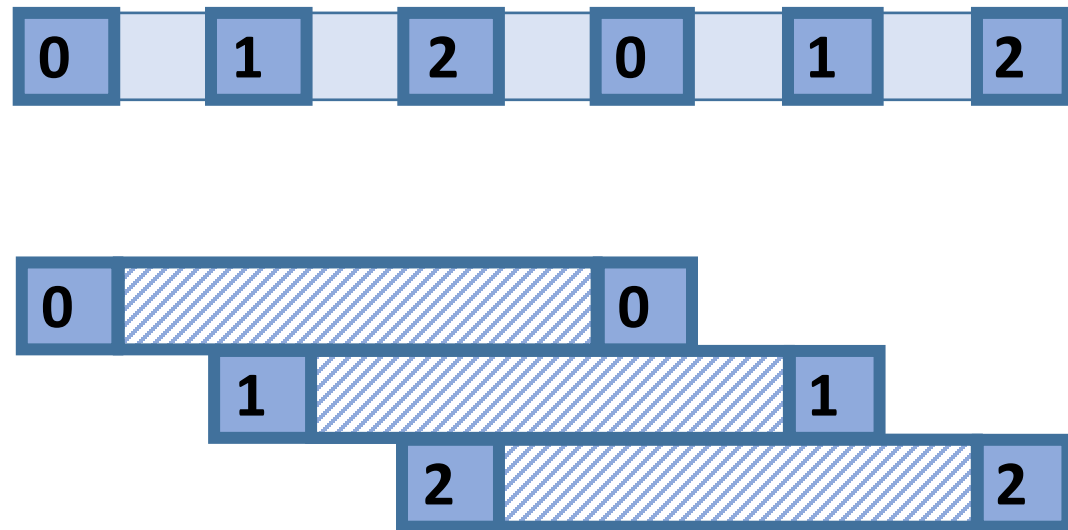
MPI_File_write_at (...)

...



Non-contiguous Access – Set File View

MPI_Init
MPI_File_open
MPI_Type_vector
MPI_Type_commit
MPI_File_set_view
MPI_File_read
MPI_File_close
MPI_Type_free
MPI_Finalize



File domain – Example

P0	P1
P2	P3

P0	P1	P0
P1	P0	P1
P2	P3	P2
P3	P2	P3

All processes read equal amounts of data

File domain

P0	P1
P2	P3

P0	P0	P0
P1	P1	P1
P2	P2	P2
P3	P3	P3

All processes read equal amounts of data
P0 and P1 exchange, P2 and P3 exchange

MPI Collective I/O

MPI_File_open (MPI_COMM_WORLD, “/scratch/largefile”,
MPI_MODE_RDONLY, MPI_INFO_NULL, &fh)

MPI_File_read_at_all (fh, offset, buffer, count, MPI_INT, status)

or

MPI_File_set_view

MPI_File_read_all (fh, buffer, count, MPI_INT, status)

MPI_File_close (&fh)

MPI Collective I/O

```
for i in `seq 1 5` ; do mpirun -np 4  
./indep 16384 ; done  
0.020404 3.063181 MB/s  
0.021844 2.861241 MB/s  
0.019550 3.196956 MB/s  
0.023124 2.702850 MB/s  
0.029048 2.151631 MB/s
```

```
for i in `seq 1 5` ; do mpirun -np 4 ./coll  
16384 ; done  
0.004425 14.124899 MB/s  
0.002833 22.062279 MB/s  
0.002439 25.627530 MB/s  
0.002959 21.123610 MB/s  
0.002194 28.481530 MB/s
```

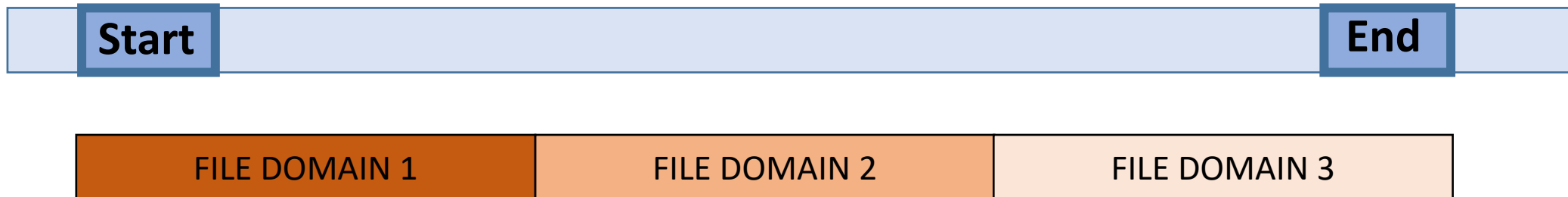
Two-phase I/O

Entire access pattern must be known before making file accesses

- Phase 1 (write)
 - Processes request for a single large contiguous chunk
 - Reduced file I/O cost due to large accesses
- Phase 2
 - Processes redistribute data among themselves
 - Additional inter-process communications

Two-phase I/O

- Phase 1
 - Processes analyze their own I/O requests
 - Create list of offsets and list of lengths
 - Everyone broadcasts start offset and end offset to others
 - Each process reads its own *file domain*



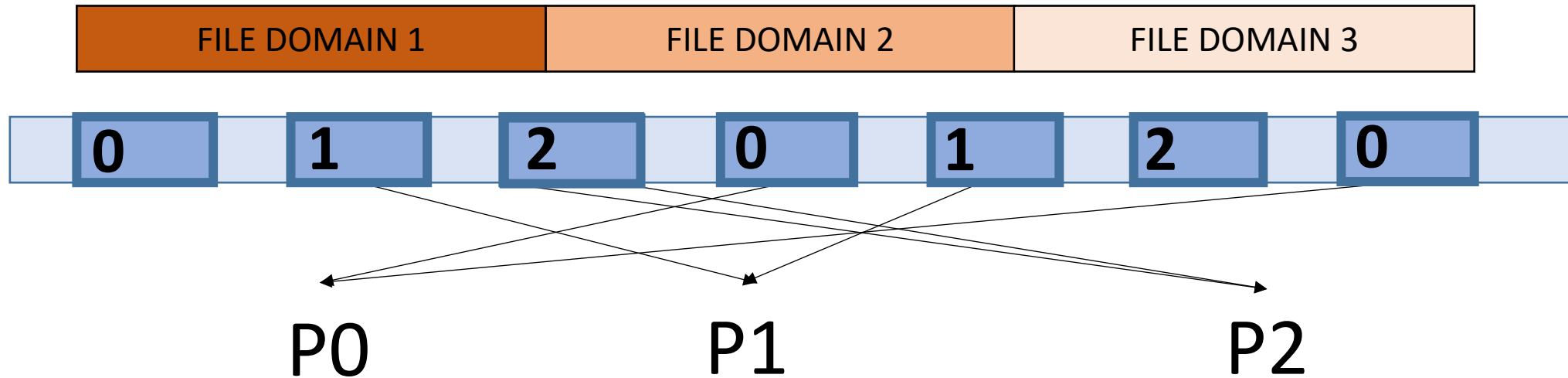
mpich/src/mpi/romio/adio/common/ad_read(write)_coll.c

```
MPI_Allgather(&start_offset, 1, ADIO_OFFSET, st_offsets, 1, ADIO_OFFSET, fd->comm);
MPI_Allgather(&end_offset, 1, ADIO_OFFSET, end_offsets, 1, ADIO_OFFSET, fd->comm);

for (i = 1; i < nprocs; i++)
    if ((st_offsets[i] < end_offsets[i - 1]) && (st_offsets[i] <= end_offsets[i]))
        interleave_count++;
```

Two-phase I/O

- Phase 2
 - Processes analyze the file domains
 - Processes exchange data with the corresponding process



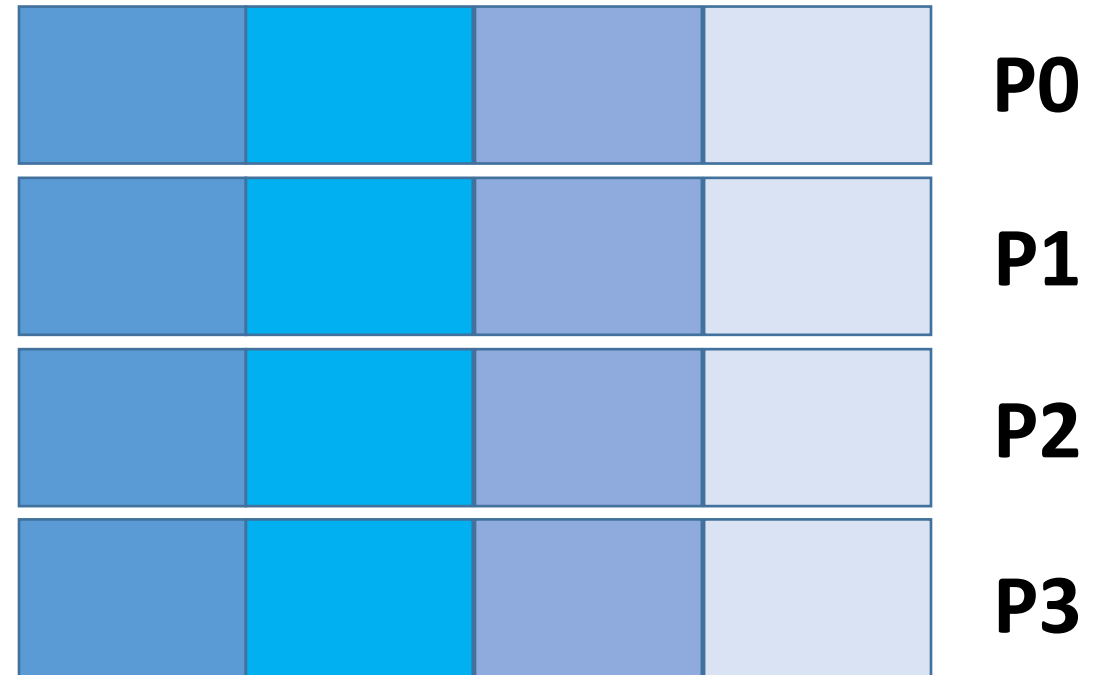
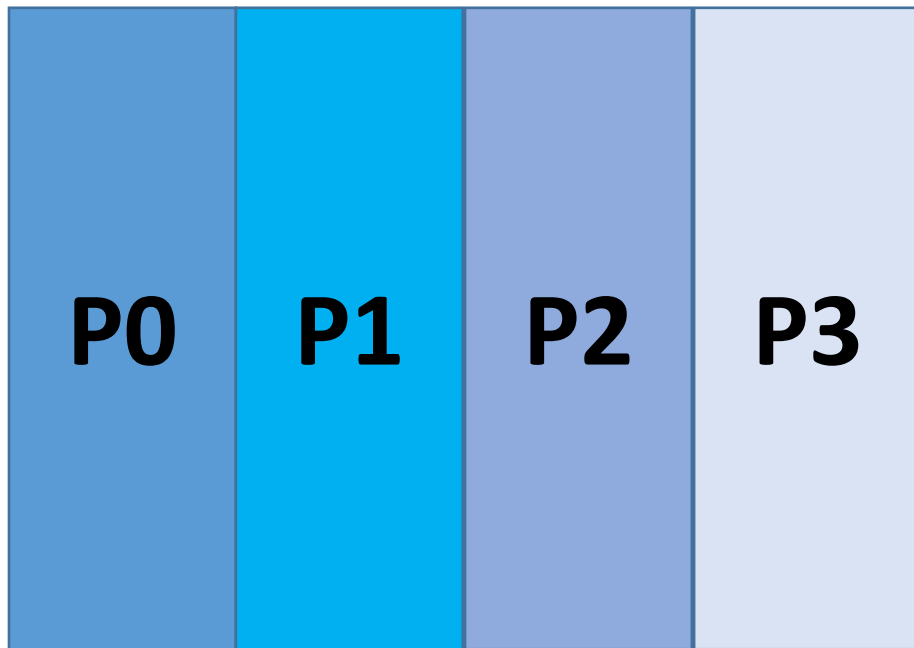
File domain – Example

P0	P1
P2	P3

P0	P0	P0
P1	P1	P1
P2	P2	P2
P3	P3	P3

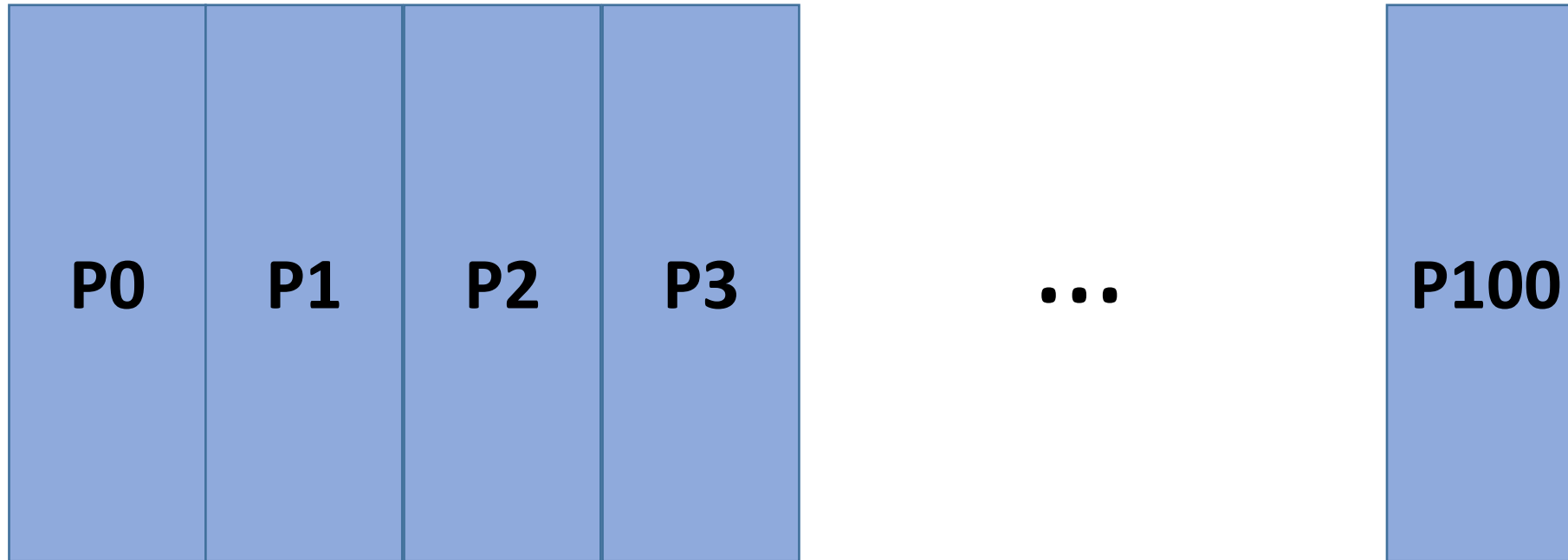
P0 and P1 exchange, P2 and P3 exchange

File domain – Example 2



Everyone needs data from every other process

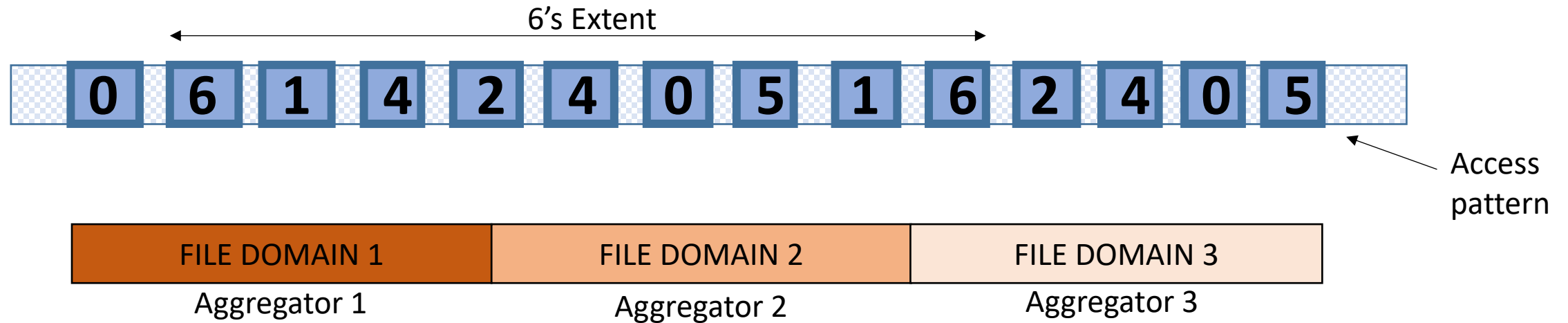
Collective I/O



Communication may become bottleneck?

Requires 100 processes participating in Alltoall communications

Collective I/O Aggregators



- Multiple small non-contiguous I/O requests from different processes are combined
- A subset of processes, I/O aggregators, access their file domains (I/O phase)
- Data redistributed among all processes (communication phase)
- Cons?

Aggregators

- Too few aggregators
 - Large buffer size required per aggregator and multiple I/O iterations
 - Underutilization of the full bandwidth of the storage system
- Too many aggregators
 - Request for large number of small chunks → suboptimal file system performance
 - Increased cost of data exchange operations

In MPICH

- Buffer size in aggregators = 16 MB
- Default number of aggregators – #unique hosts
- Placement – Specific to file system
 - `src/mpi/romio/adio/ad_gpfs/ad_gpfs_aggrs.c` (GPFS)

I/O Aggregators – Limited buffer

Total number of processes = 1024

Let each process read 2^{20} doubles (= 1 MB)

Total number of aggregators = 16

Temporary buffer in each aggregator process = 4 MB

- Collective I/O may be done in several iterations
- Double buffering may help

$$\text{I/O data size per aggregator (D)} = \frac{1024 * 1}{16} \text{ MB}$$

$$\text{Number of times each aggregator needs to do the I/O} = \frac{D}{4} = 16$$



User-controlled Parameters

- Number of aggregators
- Buffer size in aggregators

User-controlled Parameters

- Number of aggregators (cb_nodes, default=1 per node)
- Placement of aggregators (cb_config_list)
- Buffer size in aggregators (cb_buffer_size, default=16MB)
- ...

- Can be set via hints (ROMIO_HINTS file)
- MPI_Info object is used to pass hints

Set Hints via MPI_Info

```
MPI_Info_create(&info);
```

```
MPI_Info_set(info, "cb_nodes", "8");
```

```
MPI_File_open(MPI_COMM_WORLD, filename, amode, info, &fh);
```

Set Hints via Environment Variable

- export ROMIO_HINTS=\$PWD/romio_hints

```
romio_cb_read enable  
romio_cb_write enable  
ind_rd_buffer_size 4194304  
ind_wr_buffer_size 1048576  
cb_config_list=*:1
```

MPIIO Hints

```
MPI_File_open (MPI_COMM_WORLD, "/pfs/datafile", MPI_MODE_CREATE |  
MPI_MODE_RDWR, MPI_INFO_NULL, &fh);  
MPI_File_get_info (fh, &info_used);  
MPI_Info_get_nkeys (info_used, &nkeys);  
for (i=0; i<nkeys; i++) {  
    MPI_Info_get_nthkey (info_used, i, key);  
    MPI_Info_get (info_used, key, MPI_MAX_INFO_VAL, value, &flag);  
    printf ("Process %d, Default: key = %s, value = %s\n", rank, key, value);  
}
```

```
export ROMIO_PRINT_HINTS=1
```

Performance

```
$ mpirun -np 6 -hosts csews2:2,csews20:2,csews30:2 ./coll-IOnoncontig 10485760
```

```
key = cb_buffer_size      value = 16777216
```

```
key = romio_cb_read       value = enable
```

```
key = romio_cb_write      value = enable
```

```
key = cb_nodes            value = 3
```

36.014944 MB/s

```
$ mpirun -np 6 -hosts csews2:3,csews20:3 ./coll-IOnoncontig 10485760
```

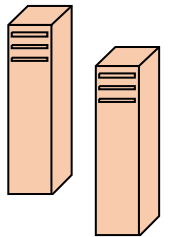
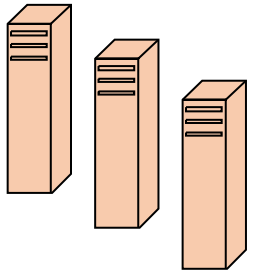
```
key = cb_buffer_size      value = 16777216
```

```
key = romio_cb_read       value = enable
```

```
key = romio_cb_write      value = enable
```

```
key = cb_nodes            value = 2
```

27.474843 MB/s



Performance

```
$ mpirun -np 6 -hosts csews2:2,csews20:2,csews30:2 ./colIOnoncontig 10485760
```

```
key = cb_buffer_size      value = 16777216
```

```
key = romio_cb_read       value = enable
```

```
key = romio_cb_write      value = disable
```

```
key = cb_nodes            value = 3
```

```
5.106368 MB/s
```

```
$ mpirun -np 6 -hosts csews2:3,csews20:3 ./colIOnoncontig 10485760
```

```
key = cb_buffer_size      value = 16777216
```

```
key = romio_cb_read       value = enable
```

```
key = romio_cb_write      value = disable
```

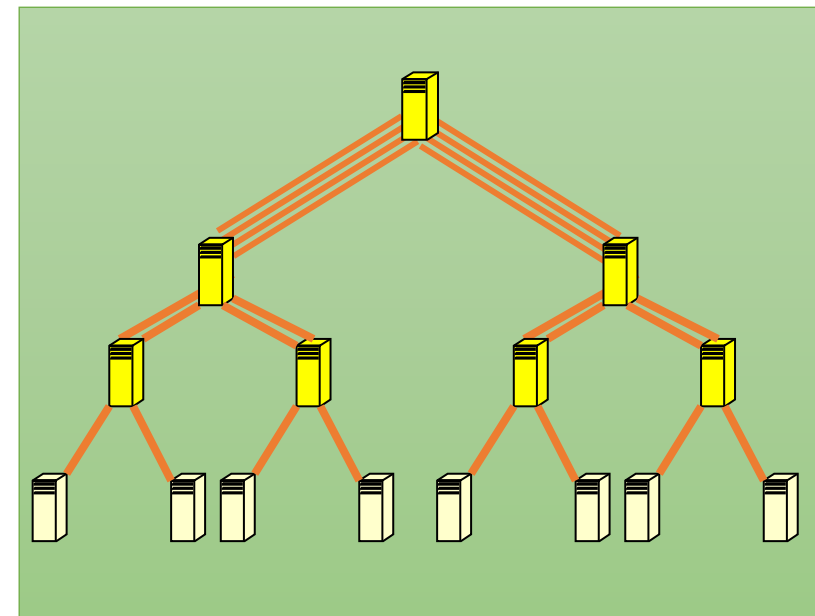
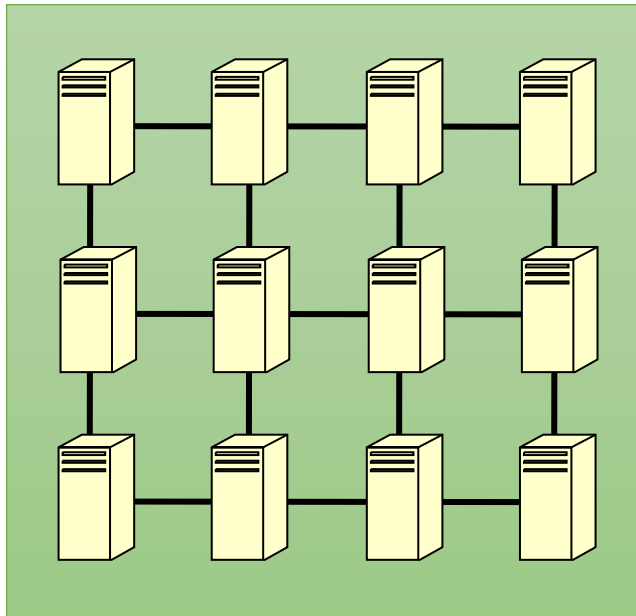
```
key = cb_nodes            value = 2
```

```
6.028663 MB/s
```


Aggregator Selection

Number of aggregators and their placements

- Depends on the architecture, file system, data size, access pattern
- Depends on the network topology, number of nodes and node placements



Multiple Non-contiguous Accesses

P0	P1	P2
P3	P4	P5

Can some of these requests be merged?

What is the access pattern?

P0 P1 P2 P0 P1 P2 P0 P1 P2 P3 P4 P5 P3 P4 P5 P3 P4 P5

Create Sub-groups for Actual I/O

P0	P1	P2
P3	P4	P5

Can some of these requests be merged?

P0 P1 P2 P0 P1 P2 P0 P1 P2 P3 P4 P5 P3 P4 P5 P3 P4 P5

P0 P3

Create Sub-groups for Actual I/O

P0 P1 P2 P0 P1 P2 P0 P1 P2 P3 P4 P5 P3 P4 P5 P3 P4 P5
P0 P3

Two leaders created
Data per process = 1 MB

P0 P1 P2 P0 P1 P2 P0 P1 P2 P3 P4 P5 P3 P4 P5 P3 P4 P5
Assign more leaders

Data per process = 1 GB

P0 P1 P2 P3 P4 P5 P0 P1 P2 P3 P4 P5 P6 P7 P8 P9 P10 P11 P6 P7 P8 P9 P10 P12
P0 and P3P6 and P9.....

Non-blocking I/O

```
MPI_Request request;
```

```
MPI_File_iwrite_at (fh, offset, buf, count, datatype, &request);
```

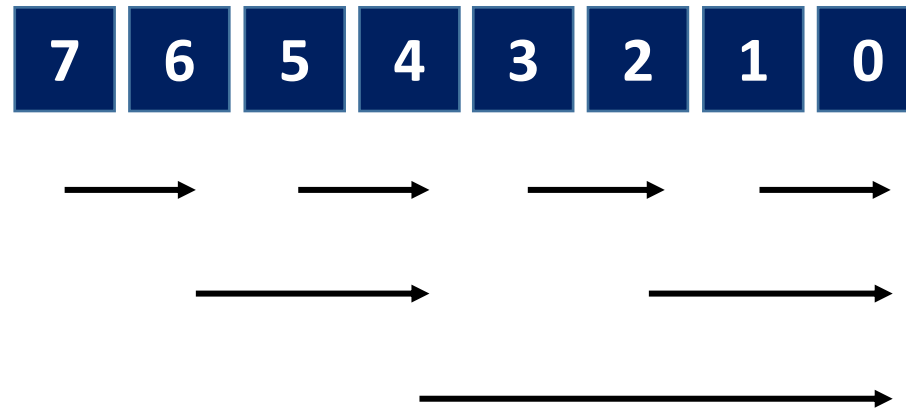
```
MPI_File_iwrite_at_all (fh, buf, count, datatype, &request);
```

```
...
```

```
/* computation */
```

```
MPI_Wait (&request, &status);
```

MPI_Gather Recursive Doubling Algorithm



Vector doubling
Distance doubling

```
CS633 $ time mpirun -np 12 --hosts csews98:6,csews99:6 ./gather 10000000
```

```
CS633 $ time mpirun -np 12 --hosts csews98:4,csews99:4,csews100:4 ./gather 10000000
```

```
CS633 $ time mpirun -np 16 --hosts csews102:8,csews101:8 ./gather 10000000

real    0m6.244s
user    0m0.031s
sys     0m0.011s
CS633 $ time mpirun -np 16 --hosts csews102:8,csews101:8 ./gather 10000000

real    0m6.018s
user    0m0.039s
sys     0m0.006s
CS633 $ time mpirun -np 16 --hosts csews12:8,csews11:8 ./gather 10000000

real    0m6.321s
user    0m0.033s
sys     0m0.005s
CS633 $ time mpirun -np 16 --hosts csews12:8,csews11:8 ./gather 10000000

real    0m6.021s
user    0m0.036s
sys     0m0.005s
```

```
CS633 $ time mpirun -np 16 --hosts csews12:6,csews11:6,csews21:4 ./gather 10000000
```

```
real    0m8.674s  
user    0m0.051s  
sys     0m0.011s
```

```
CS633 $ time mpirun -np 16 --hosts csews12:6,csews11:6,csews21:4 ./gather 10000000
```

```
real    0m8.763s  
user    0m0.058s  
sys     0m0.014s
```

```
CS633 $ time mpirun -np 16 --hosts csews102:6,csews11:6,csews201:4 ./gather 10000000
```

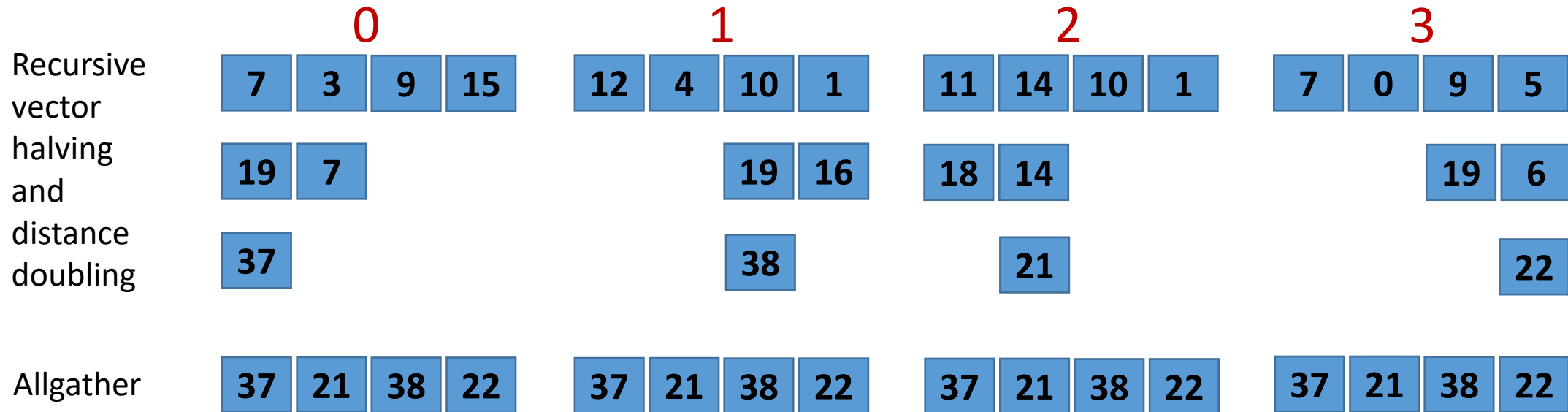
```
real    0m17.254s  
user    0m0.052s  
sys     0m0.012s
```

```
CS633 $ time mpirun -np 16 --hosts csews102:6,csews11:6,csews201:4 ./gather 10000000
```

```
real    0m16.522s  
user    0m0.062s  
sys     0m0.011s
```


Errata in earlier slides

Allreduce – Rabenseifner’s Algorithm



Time:

$\log p * L + (p-1)/p * (n/B) + (p-1)/p * n * c$ (reduce-scatter) + doubling

$\log p * L + (p-1)/p * (n/B)$ (allgather using recursive vector doubling and distance halving)

n = data size

L = latency

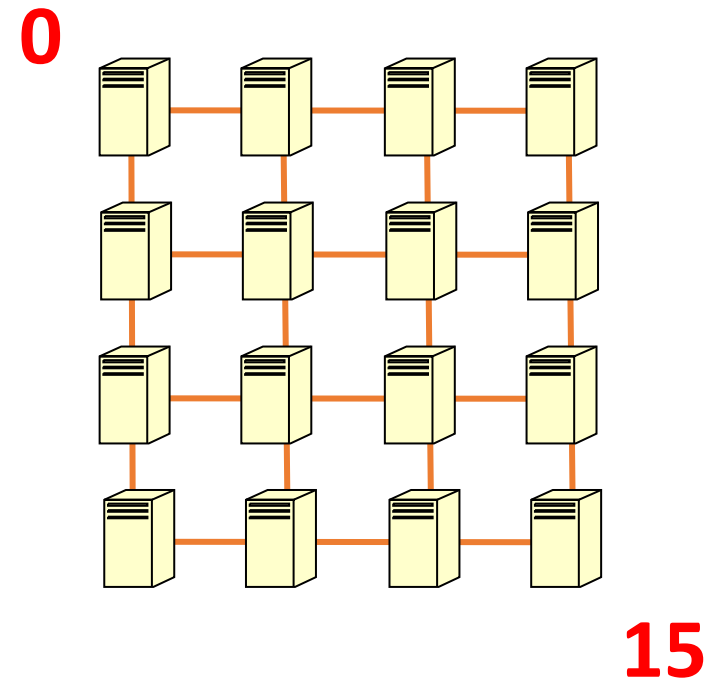
p = #processes

B = bandwidth

c = compute cost per byte

MPI_Gather Communications (Hops, Volume)

#1	$0 \leftarrow 1 (1, n)$	$8 \leftarrow 9 (1, n)$
	$2 \leftarrow 3 (1, n)$	$10 \leftarrow 11 (1, n)$
	$4 \leftarrow 5 (1, n)$	$12 \leftarrow 13 (1, n)$
	$6 \leftarrow 7 (1, n)$	$14 \leftarrow 15 (1, n)$
#2	$0 \leftarrow 2 (2, 2n)$	$4 \leftarrow 6 (2, 2n)$
	$8 \leftarrow 10 (2, 2n)$	$12 \leftarrow 14 (2, 2n)$
#3	$0 \leftarrow 4 (1, 4n)$	$8 \leftarrow 12 (1, 4n)$
#4	$0 \leftarrow 8 (2, 8n)$	



Given: Message size at every process = n ,
default XY process placement

One-to-many Non-Blocking Communications

~~All ranks except 0 send to rank 0~~

rank 0 sends to all other ranks

// Write the code (H.W.)

// Q: What is missing? A: MPI_Wait

if I am rank 0

 for (r = 1; r < size; r++)

 MPI_Isend (to r)

else

 MPI_Recv (from 0)